QTD-HGNN - Technical Notes

Every data field, what it means, how it is stored, and how the system works

QTD-HGNN - Quantum-Topological Threat Correlation

Problem Statement 2: AI-Driven Correlation of Cybersecurity Telemetry & Transactional Behaviour

1. What the system is

QTD-HGNN correlates cybersecurity telemetry (logins, devices, IPs, TLS posture) with transactional behaviour (payments, payees, channels) to detect account takeover, mule rings, and quantum-risk indicators - with explainable, auditable alerts. Every number shown is computed by a trained model and a real pipeline; nothing is simulated.

Core thesis: a suspicious login and a large transfer to a new payee are each unalarming alone, but together = account takeover. The SIEM sees the first, the fraud engine sees the second, neither sees the attack. We fuse both domains in one model.

2. The six PS2 outcomes -> what delivers each

Expected outcome	What delivers it
Correlate cyber + transaction	Heterogeneous graph fuses login/device/TLS with payment features per customer
Detect threats proactively	Trained GraphSAGE scores each transaction; ATO kill-chains surface in real time
Identify fraud patterns	Same-customer/same-payee graph edges + persistent-homology (Betti) structure
Quantum-related indicators	Crypto-posture: quantum-vulnerable TLS %, downgrade events (honest, observable)
Reduce false positives	Cross-domain corroboration + tuned decision threshold (precision 0.71 @ recall 0.98)
Explainable AI	Per-alert SHAP reason codes (auditable, RBI/DPDP-aligned)

3. Architecture - 5-stage pipeline

```
Ingest        Fuse              Detect          Explain      Act
cyber+txn --> graph+topology --> GraphSAGE  --> SHAP     --> SOC dashboard
(logins,      (Ripser Betti     (16 fused      (subgraph    (alerts, threat
 payments)     b0/b1/b2)         features)      reason        score, quantum,
                                                codes)        model insights)
```

- **Layers:** ml/ - synthetic data generator, feature engineering + persistent homology, GNN training + evaluation, build-time pre-scoring.
- backend/ - FastAPI + SQLAlchemy/Alembic; loads the trained model, serves REST + a WebSocket telemetry stream, persists auditable alerts.
- frontend/ - React/Vite SOC dashboard (Tailwind + shadcn), bound entirely to the live backend.

4. Input data - the synthetic bank

No public dataset contains BOTH cyber telemetry and transactions for the SAME entities with labels (confirmed - production SOC+payment data cannot be released for privacy). The defensible standard approach: generate one shared customer population, attach both domains, and inject labelled attack scenarios. Scale: ~6,000 customers, 559,661 transactions, 445,158 logins over a 45-day window. The ingestion format is designed to swap in real bank feeds unchanged.

4.1 Entities (static population)

Table / field	Meaning
customers.customer_id	Unique customer id (C000000...)
customers.home_city	Home city id (0-39) - baseline geo
customers.login_rate / txn_rate	Per-customer mean logins & transactions per day (behaviour baseline)
customers.amount_mu / amount_sigma	Lognormal params for this customer's normal transaction amount
accounts.account_id	Primary account per customer (A000000...)
accounts.opened_day	Day account opened (relative to sim window)
devices.device_id / usual_city	Known device per customer + its usual city
payees.payee_id / risk_flag	Beneficiary id; ~3% pre-flagged risky

4.2 Cyber telemetry - login events

Field	Meaning
login_id, customer_id	Event id + owning customer
t_day, hour	Timestamp (fractional day) and hour-of-day
device_id, city	Device used and its city
new_device	1 if device never seen for this customer (ATO signal)
failed_prior	Failed login attempts before this success (credential stuffing signal)
tls	Negotiated TLS/cipher, e.g. TLS1.3:X25519MLKEM768 (strong) or TLS1.0:RSA-EXPORT (weak)
label, scenario	Ground truth: benign/malicious + scenario tag

4.3 Transactional behaviour - transaction events

Field	Meaning
txn_id, customer_id, account_id	Transaction id + owning customer/account
t_day	Timestamp (fractional day)
amount	Transaction value (INR)
channel	UPI / NEFT / IMPS / CARD / SWIFT
payee_id, new_payee	Beneficiary; 1 if payee is new (ATO/mule signal)
sec_since_login	Seconds between the preceding login and this transaction
sec_since_payee_add	Age of the payee relationship (fresh payee = risk)
label, scenario	Ground truth label + scenario tag

4.4 Injected labelled scenarios (the ground truth)

- **Account takeover:** Hostile login (new device + geo, failed-login burst, weak TLS) -> new payee -> large drain, seconds apart. ~220 episodes.
- **Mule ring:** A ring of accounts passing funds in a cycle through a shared hub (layering). ~40 rings.
- **HNDL indicator:** Crypto-downgrade login (weak/export cipher) + bulk-egress staging transaction (huge, SWIFT, off-hours). ~120 chains.
- **Benign anomaly:** Legitimate-but-unusual events (a real large purchase, a genuine new phone) - LABELLED BENIGN so the model learns NOT to flag them. ~400. This is what trains false-positive suppression.

5. Feature engineering - the fusion row (16 features)

For each transaction, the nearest preceding login is joined in, and cross-domain features are computed. Cyber columns and transaction columns sit in the SAME row - the thing no siloed bank tool builds. The model consumes these 16 features:

Feature	Domain / meaning
amount_zscore	Txn: amount vs this customer's normal (std deviations)
log_amount	Txn: log transaction size
new_payee	Txn: payee is new
sec_since_login, sec_since_payee_add	Txn: timing - login->txn gap, payee age
is_swift, is_imps	Txn: channel flags
login_new_device	Cyber: login from an unseen device
login_failed_prior	Cyber: prior failed-login count
login_odd_hour	Cyber: login in 00:00-06:00
tls_quantum_vulnerable	Cyber/quantum: RSA/ECDHE-RSA (not PQC-hybrid)
tls_weak_downgrade	Cyber/quantum: export/weak cipher (downgrade indicator)
geo_mismatch	Cross: geo inconsistency signal
cust_betti0 / betti1 / betti2	Topology: shape of this customer's transaction cloud

> **The fixed disconnect**
> In the discarded prototype, Betti numbers were computed but NEVER fed to the model, and were faked as 'variance + random'. Here they are real Ripser persistent homology AND are actual model features (cust_betti0/1/2).

6. Topology - persistent homology (real)

A point cloud of feature vectors is built and a Vietoris-Rips filtration is computed with Ripser. Betti numbers count holes that persist across scales: b0 = connected components (clusters), b1 = loops (ring-like structure - money-mule signal), b2 = voids. Validated: a malicious feature cloud shows b1 = 18 loops vs 0 for benign. On the live dashboard, Betti is computed on a rolling buffer of the most-anomalous points so ring structure accumulates and b1 loops surface during attacks.

7. The model - GraphSAGE GNN

- **Graph:** Nodes = transactions. Edges connect transactions sharing a customer (temporal sequence -> ATO) and sharing a payee (hub -> mule ring).
- **Architecture:** 2-layer GraphSAGE, 16 input features (incl. topology) -> 2 classes. Class-weighted for the ~0.1% fraud rate; train/test split; best-F1 checkpoint; decision threshold tuned on train predictions to cut false positives.

Held-out test metrics (real, not inflated)

Metric	Value	Meaning
Recall	0.977	catches 97.7% of real fraud
Precision	0.714	71% of alerts are true (~29% FP - vs 90%+ cited in current SOCs)
F1	0.825	balance of the two
AUC	0.99998	near-perfect ranking
Threshold	0.99	tuned; 559,661 txns scored, 776 flagged

8. Explainability - SHAP

Every flagged transaction is explained by computing SHAP values on the node's local 2-hop subgraph (fast, ~0.4s, real values on the actual trained model). Positive values push toward malicious, negative toward benign. Example for a real account-takeover: amount_zscore +0.52, login_new_device +0.22, is_swift +0.09. These become plain-English reason codes - auditable per RBI Fraud-Risk & DPDP.

9. How data is stored

- **Stores:** PostgreSQL (prod) / SQLite (local), via SQLAlchemy ORM; schema managed by Alembic migrations. Raw synthetic data lives as CSV; the trained model as model.pt + scaler; the pre-scored dataset as scored.csv + graph.pt (built into the image).

The Alert table (one row per flagged transaction - auditable)

Column	Meaning
id	Primary key
txn_id, customer_id	Which transaction / customer
amount	Transaction value
threat_score	0-100 (model probability x 100)
predicted_label	0 benign / 1 malicious
scenario	Ground-truth tag (synthetic): account_takeover / mule_ring / hndl_indicator / ...
reason_codes	JSON: [{feature, value, shap_value}] - the SHAP explanation
created_at	Timestamp (indexed)

> **Why a real DB matters**
> Under RBI + DPDP a bank cannot act on a black-box score. Persisting each alert with its SHAP reason codes makes every decision reproducible and legally defensible - versus the discarded prototype's ephemeral random JSON.

10. What the dashboard shows - every field

Panel / field	Source & meaning
Threat Score (KPI + trend)	Mean of the top-20 window probabilities x100 - severity of riskiest activity
Active Threats	Count of transactions >= 0.99 in the current window
Transactions Scanned	Transactions in the current 0.5-day window
Connection	Live WebSocket status to the PyTorch backend
Betti panel (b0/b1/b2 + curve)	Real Ripser homology on the rolling anomaly buffer
Live Alert Queue	Flagged transactions: scenario badge, amount, confidence
SHAP waterfall	On-click: real per-feature Shapley attributions for that alert
Quantum Risk Posture	% quantum-vulnerable TLS, downgrade events, modern % (from window logins)
Model Insights	Real held-out precision/recall/F1/AUC from metrics.json

11. Runtime data flow

- **Build time:** Data generated + whole dataset scored -> scored.csv + graph.pt baked into the image. Makes startup instant and the container self-contained.
- **Startup + stream:** Model + pre-scored artifacts loaded (~0.8s). Each 1.5s tick slices the pre-scored rows for the next time window and broadcasts over WebSocket. No heavy compute on the event loop (that previously hung the server).
- **On demand:** When an analyst clicks an alert, GET /explain/{txn_id} computes SHAP on the node's subgraph off the event loop (~0.4s), cached.

12. Honesty notes

Data is synthetic (no public dataset joins both domains), with a swappable ingestion format. The quantum module reports real cryptographic posture and observable HNDL indicators (crypto-downgrade, bulk egress); it does NOT claim to detect passive interception, which is physically undetectable - the mitigation is PQC migration. Topology, the trained GNN, SHAP, and all metrics are genuine and reproducible.